

통합 리눅스 서비스 이관 실습 교재

이론·큰그림·실습 명령어·검증·장애 대응을 한 권으로 이해하기

이 문서의 목적

서버를 처음 보는 학생도 현재 서버 구조를 머릿속에 그릴 수 있도록, 어려운 용어를 집과 이사에 비유해 설명한다. 명령어를 실행하는 문서가 아니라, 실행 전에 무엇을 확인해야 하는지 이해하는 문서다.

이번 연습의 기준

기존 1번 PDF의 이론을 바탕으로, 팀장님이 설명한 VM 2번 -> VM 1번 서비스 재구성 작업을 추가했다. VM 전체를 복사하는 것이 아니라 필요한 디렉터리, 라이브러리, 환경변수, 서비스 연결과 클러스터를 직접 구성하는 연습이다.

핵심 문장

VM은 복사해서 운반하고, 데이터는 서비스별 안전한 방식으로 복원하고, 트래픽은 마지막에 조금씩 연결한다.

문서 기준일: 2026-09-03 / 설명 수준: 개발 공부하는 고등학생

0. 이 문서를 읽는 순서

지금 가장 먼저 알아야 할 것은 명령어가 아니라 '무엇이 무엇에 연결되어 있는가'입니다. 컴퓨터를 잘 몰라도 괜찮습니다. 아래 순서로 읽으면 됩니다.

- 1단계: Xen 호스트, VM, OS를 구분한다.
- 2단계: 서비스, 프로세스, 포트를 구분한다.
- 3단계: CMS 화면이 HAProxy와 여러 백엔드 서비스에 의존한다는 것을 이해한다.
- 4단계: Redis, Elasticsearch, Solr, MariaDB의 데이터 성격이 서로 다르다는 것을 이해한다.
- 5단계: VM 복사 후 바로 운영에 연결하지 않고 target을 격리하는 이유를 이해한다.
- 6단계: 마지막에 L4 또는 특정 IP로 조금씩 연결하는 이유를 이해한다.

중요한 약속

이 문서에 나오는 명령어는 이해를 위한 예시다. start, stop, restart, rm, firewall 변경, 데이터 삭제 명령은 승인 없이 실행하지 않는다.

1. 이관이란 무엇인가

이관은 단순히 IP 주소를 바꾸는 일이 아닙니다. 기존 환경에서 돌아가던 컴퓨터와 프로그램을 새 환경으로 옮긴 뒤, 새 주소와 새 연결관계에서도 똑같은 업무가 되는지 확인하는 일입니다.

1.1 집 이사로 이해하기

서버 개념	10살 어린이식 비유	실제 의미
Xen 호스트	아파트 건물	VM 여러 대가 올라가는 물리 또는 가상화 호스트
VM	아파트 한 채	OS, 프로그램, 디스크를 가진 독립된 가상 컴퓨터
OS	집의 바닥과 벽	Rocky Linux 같은 운영체제
서비스	집 안의 가전제품	MariaDB, Redis, Solr, Elasticsearch 같은 서버 프로그램
데이터	서랍 속 문서	DB 행, 검색 index, Solr collection, Redis key
IP	집 주소	네트워크에서 서버를 찾는 숫자 주소
Port	집의 초인종 번호	어떤 프로그램으로 들어갈지 구분하는 번호
L4/VIP	아파트 안내 데스크	요청을 어느 서버로 보낼지 결정하는 앞단

따라서 VM을 복사해도 다음은 별도로 확인해야 합니다.

VM 자체가 새 호스트에 존재하는가
↓
OS와 네트워크가 정상인가
↓
프로그램 파일과 설정이 있는가
↓
DB, 검색 데이터, cache가 올바른 방식으로 복원됐는가
↓
애플리케이션이 target 서비스 주소를 보고 있는가
↓
테스트 요청을 받아 업무가 실제로 되는가

현재 환경에서 특히 주의할 점

첨부 런북은 VMware/Oracle Linux 9.6을 전제로 하지만, 우리가 확인한 현재 VM은 Xen 환경의 Rocky Linux 9.6이다. 그러므로 런북의 명령어를 그대로 붙여 넣지 말고, 원칙과 검증 순서를 가져와 Xen에 맞는 절차로 다시 작성해야 한다.

2. Xen 구조: 호스트, VM, Pool, SR

Xen을 처음 보면 '서버 안에 서버가 또 있다'는 것이 가장 헷갈립니다. 실제로는 큰 컴퓨터인 호스트 위에 여러 개의 가상 컴퓨터인 VM이 올라가 있습니다.



2.1 Pool은 무엇인가

Pool은 여러 Xen 호스트를 하나의 관리 그룹처럼 묶은 것입니다. 'pool에 들어 있다'고 해서 VM 데이터가 자동으로 모든 호스트에 복사된다는 뜻은 아닙니다. 관리 화면에서 함께 보이고, 조건이 맞으면 VM 이동이나 관리 작업을 쉽게 할 수 있는 단위입니다.

2.2 SR은 무엇인가

SR은 Storage Repository의 줄임말입니다. VM의 가상 디스크를 넣어 두는 창고입니다. VM의 CPU/RAM 설정과 디스크 데이터는 별개의 정보처럼 보이지만, 이관에서는 둘 다 필요합니다.

용어	쉽게 말하면	이관 때 확인할 것
Host	VM을 실행하는 큰 컴퓨터	CPU, RAM, 네트워크, Xen 관리 권한
Pool	호스트들의 관리 그룹	source와 target이 같은 pool인지, 이동 조건
SR	VM 디스크 창고	용량, 연결 방식, target에서 보이는지
VM	독립된 가상 컴퓨터	CPU, RAM, 디스크, NIC, UUID, 자동기동

현재 이관에서 기억할 구조

이번 목적지는 tc-xen01 호스트다. tc-xen01은 큰 컴퓨터이고, 그 안에 xen01.01(192.168.1.81), xen01.02(192.168.1.82), xen01.03(192.168.1.83) 같은 VM이 있다. 호스트 주소와 VM 주소는 서로 다르다. tc-xen01로 접속했다고 Qdrant나 Elasticsearch가 바로 실행되는 것은 아니다.

2.3 이번 목적지에 이미 있는 VM 이해하기

tc-xen01에는 이미 사용할 수 있는 VM이 있고, 그 안에 서비스가 설치되어 있다는 자료가 있습니다. 그러므로 목적지는 빈 교실이 아니라 책상과 물건이 이미 있는 교실입니다. 기존 물건을 지울지, 옆에 새 교실을 만들지, 기존 물건에 새 자료를 넣을지는 담당자의 결정이 필요합니다.

목적지 VM	주소	이미 기록된 서비스	이론적으로 중요한 점
xen01.01	192.168.1.81	Qdrant 01 / v1.15.0	Qdrant collection은 벡터 검색 데이터이므로 보존·복원 방법을 따로 정함
xen01.02	192.168.1.82	Elasticsearch 9.1.1 / 9200-9300	Elasticsearch index는 data 폴더를 바로 복사하지 않고 Snapshot/Restore를 우선 생각함
xen01.03	192.168.1.83	PostgreSQL 5432 / Gitea 3000	DB, Git 저장소, 설정파일, 첨부파일, 비밀키가 함께 맞아야 함

절대 섞어 생각하지 않기

VM 전체 복사는 컴퓨터 한 대를 복사하는 일이고, 서비스 데이터 복원은 그 컴퓨터 안의 DB·검색·저장소를 제품이 이해하는 방법으로 옮기는 일이다. 목적지 VM이 이미 사용 중이면 원본 VM 디스크를 그대로 덮어쓰는 것은 단순 복사가 아니라 기존 환경 삭제를 포함할 수 있다.

3. 서버를 7층으로 나누어 보기

서버에서 문제가 생겼을 때 'CMS가 안 돼요'라고만 말하면 원인을 찾기 어렵습니다. 아래 7층 중 어느 층이 문제인지 나누어 생각해야 합니다.

층	질문	예시
1. VM/OS	어느 컴퓨터이고 어떤 OS인가?	VM 101, Rocky Linux 9.6, 5 vCPU
2. Network	주소와 길이 맞는가?	IP, gateway, DNS, route, hosts
3. Runtime	실행에 필요한 기본 도구가 있는가?	Java, Python, JDK, native library
4. Service/Process	프로그램이 실제로 실행 중인가?	java -jar CMS, redis-server, Solr
5. Config	프로그램이 어떤 주소를 보도록 설정됐는가?	yaml, properties, conf, env
6. Data	업무 데이터가 어디에 있는가?	MariaDB table, ES index, Solr collection
7. Connection	다른 서비스와 통신되는가?	CMS -> Master -> DB/Redis/Solr/ES

이 7층을 순서대로 보면, CMS가 안 될 때 다음처럼 좁혀갈 수 있습니다.

VM이 켜져 있는가
↓
IP와 route가 맞는가
↓
CMS Java 프로세스가 있는가
↓
CMS 포트가 LISTEN인가
↓
방화벽이 포트를 막지 않는가
↓
HAProxy가 올바른 Host 이름을 받는가
↓
CMS가 Master/DB/Redis 등과 통신하는가

4. 서비스, 프로세스, 포트, 설정, 데이터

이 다섯 단어를 섞지 않는 것이 중요합니다. 예를 들어 'CMS가 있다'는 말은 파일만 있다는 뜻인지, 실행 중인지, 외부에서 접속 가능한지에 따라 전혀 다른 상태입니다.

대상	의미	확인 질문
파일	실행할 수 있는 재료	cms-0.0.1-SNAPSHOT.jar가 있는가?
프로세스	메모리에서 실제 실행 중인 프로그램	java -jar CMS가 실행 중인가?
포트	프로그램이 요청을 받는 번호	8280에서 LISTEN 중인가?
설정	프로그램이 사용할 주소와 옵션	DB/Redis/Solr/ES 주소가 어디인가?
데이터	프로그램이 읽고 쓰는 업무 내용	DB table, index, collection이 맞는가?

예를 들어 CMS는 다음처럼 구성됩니다.

파일: /application/cms/cms-0.0.1-SNAPSHOT.jar
↓
프로세스: /usr/bin/java -jar .../cms-0.0.1-SNAPSHOT.jar
↓
내부 포트: 8280
↓
앞단: HAProxy가 80번 포트에서 Host 이름을 보고 CMS backend로 전달
↓
연결 대상: Master, DB, Redis, Solr, Elasticsearch 등

파일이 있다고 서비스가 정상인 것은 아니다

파일만 있으면 '재료가 있다'는 뜻이다. 프로세스가 실행되어야 하고, 포트가 열려야 하며, 설정이 맞아야 하고, 의존 서비스와 통신되어야 실제 CMS가 된다.

5. 현재 확인한 CMS 접속 구조

이번 분석에서 실제로 확인한 CMS 접속은 직접 8280으로 들어가는 방식이 아니라, HAProxy를 거치는 방식이었습니다.



5.1 왜 IP:8280으로 직접 접속하지 않았나

CMS 애플리케이션은 8280에서 실행되지만, 서버 방화벽이 8280을 외부에서 직접 접근하지 못하게 막고 있었습니다. 서버 내부에서 127.0.0.1:8280으로 요청했을 때 401 Unauthorized가 반환되었으므로, CMS 자체는 살아 있고 인증이 필요하다는 것까지 확인했습니다.

5.2 401 Unauthorized는 무엇인가

401은 '서버가 죽었다'가 아닙니다. '요청은 도착했지만 로그인하지 않았으므로 허용하지 않는다'는 응답입니다. 반대로 connection refused나 timeout은 포트·프로세스·네트워크 문제일 가능성이 큼니다.

브라우저/명령 결과	쉬운 의미
HTTP 200	요청을 정상 처리함
HTTP 301/302	다른 주소나 로그인 화면으로 이동시킴
HTTP 401	인증 필요. 서버는 응답하고 있음
HTTP 403	인증은 되었지만 권한 없음
HTTP 404	서버는 응답하지만 경로를 모름
connection refused	해당 포트에서 받을 프로그램이 없거나 거부
timeout	경로 또는 방화벽 때문에 응답을 못 받음

6. hosts 파일과 HAProxy의 차이

이번 작업에서 가장 많이 헷갈린 부분입니다. hosts 파일과 HAProxy는 서로 다른 일을 합니다.

대상	하는 일	예시
Windows hosts	이름을 IP로 바꿈	192.168.1.101 cms
HAProxy	도착한 요청을 어느 backend로 보낼지 결정	Host가 cms이면 CMS backend
CMS	실제 웹 애플리케이션	cms1/cms2/cms3:8280

예시로 `http://cms`를 입력했을 때 일어나는 일

브라우저가 cms라는 이름을 물어봄
↓
Windows hosts가 192.168.1.101이라고 알려줌
↓
192.168.1.101:80의 HAProxy가 요청을 받음
↓
HAProxy가 Host: cms를 확인함
↓
CMS backend 중 하나인 cms1/cms2/cms3:8280으로 전달

6.1 환경변수와 hosts는 다르다

형태	종류	뜻
192.168.1.101 cms	hosts 파일	cms라는 이름을 192.168.1.101로 해석
export CMS_HOST=...	환경변수	현재 프로세스가 읽을 변수 값을 설정
bind 0.0.0.0:80	HAProxy 설정	모든 IPv4 인터페이스의 80번 포트에서 대기
server cms1 cms1:8280	HAProxy backend	CMS backend 이름과 포트 지정

7. CMS, Master, Engine, Gateway, Chat UI 연결 구조

CMS는 화면이고, 뒤에서 여러 프로그램이 일을 나눠 합니다. 한 프로그램이 모든 일을 하는 것이 아니라, 역할별로 나뉘어 서로 HTTP/API와 DB 연결을 사용합니다.



7.1 런북에 적합한 애플리케이션 포트

프로그램	예시 포트	역할
gateway	8081	API 요청의 입구
engine	8180	봇/엔진 처리
cms	8280	관리자 웹 애플리케이션
master	8380	중앙 관리 API
chat-ui	8480	챗봇 사용자 화면
scheduler	8580	예약·자동 작업 실행

기동 순서의 이유

데이터 서비스가 먼저 준비되어야 애플리케이션이 연결할 곳을 찾을 수 있다. 그래서 일반적으로 MariaDB, ZooKeeper/Solr, Elasticsearch/Kibana, Redis를 확인한 뒤 master -> engine -> gateway -> CMS -> chat-ui 순서로 한 단계씩 올린다. Scheduler는 중복 실행 위험 때문에 마지막이다.

8. 서비스별 역할을 한눈에 보기

서비스	무엇을 저장/처리하나	이관 시 핵심
MariaDB	업무의 원본 데이터, 사용자, 설정, 상태	전체를 무조건 덮지 않고 writer를 멈춘 뒤 필요한 범위를 정합화
Redis	빠르게 읽는 cache와 cluster 상태	기존 cluster 재결합 우선, 실패하면 빈 cluster 후 cache 재생성
Elasticsearch	검색 index와 shard	data 폴더 직접 복사 대신 Snapshot/Restore
Solr	collection, shard, replica, 검색 문서	모든 collection을 BACKUP/RESTORE, configset/alias 확인
ZooKeeper	SolrCloud의 설정과 멤버십 조정	clean ensemble 구성, 오래된 내부 상태 원시 복사 금지
Qdrant	벡터 검색 collection	collection과 storage, 애플리케이션 연결 확인

중요한 구분

- MariaDB는 업무의 기준이 되는 데이터가 많으므로 함부로 비우면 안 된다.
- Elasticsearch와 Solr는 폴더만 옮기는 방식으로 cluster 일관성을 보장할 수 없다.
- Redis는 원본 데이터가 아니라 재생성 가능한 cache라는 전제를 둔다.
- ZooKeeper는 Solr의 조정자이지 검색 문서 자체를 보관하는 장소로 보면 안 된다.
- CMS가 쓰더라도 뒤의 DB와 검색 서비스가 틀리면 실제 업무는 실패한다.

9. Redis: 왜 재생성 가능하다고 하는가

Redis는 메모리를 이용해 아주 빠르게 값을 주고받는 저장소입니다. 이번 런북에서는 Redis를 주로 cache로 봅니다. cache는 원본 문서라기보다 '빨리 쓰려고 잠시 만들어 둔 복사본'에 가깝습니다.

9.1 cache를 지워도 되는 경우

원본이 MariaDB, Solr, Elasticsearch 또는 애플리케이션의 배포 기능에 있고, Redis 값이 그 원본에서 다시 계산될 수 있다면 Redis cache는 다시 만들 수 있습니다. 그러나 모든 Redis가 항상 cache인 것은 아니므로 실제 사용처 확인 없이 삭제하면 안 됩니다.

9.2 Redis cluster를 두 갈래로 처리

경로	무엇을 하나	성공 기준
우선 경로	미러의 정적 설정, cluster state, AOF/RDB가 서로 맞으면 기존 cluster 재결합	cluster_state ok, 16,384 slot 정상, master 3/replica 3, known node 6
대체 경로	상태가 꼬이면 기존 동적 파일을 hold로 보관하고 빈 cluster를 새로 구성	새 node ID와 slot, master/replica 관계가 정상
cache 재생성	시스템 배포 기능으로 필요한 cache를 다시 생성	대표 key의 type/value/TTL과 애플리케이션 조회 정상

9.3 7000/7001은 무엇인가

실제 환경에서는 Redis 인스턴스가 7000과 7001 포트로 나뉘어 있습니다. 다만 '7000이 항상 master'라고 미리 가정하면 안 됩니다. cluster nodes와 replication 정보를 보고 현재 역할을 확인해야 합니다.

절대 섞으면 안 되는 것

Redis의 정적 설정 파일과 Redis가 자동으로 관리하는 cluster state 파일은 이름이 비슷할 수 있어도 역할이 다르다. cluster state만 지우고 AOF/RDB를 남기면 새 cluster 생성이 막힐 수 있다.

10. Elasticsearch: 왜 Snapshot/Restore인가

Elasticsearch는 여러 노드가 index를 shard로 나누어 보관합니다. 각 노드는 cluster UUID, node ID, shard, translog 같은 내부 상태도 가집니다. 여러 노드의 data 디렉터리를 서로 다른 시점에 복사하면 하나의 일관된 순간을 보장하기 어렵습니다.

10.1 안전한 개념



10.2 global state를 조심하는 이유

Snapshot에는 index 데이터 외에도 cluster 설정이 포함될 수 있습니다. source에서 사용하던 allocation filter, read-only block, recovery throttle 같은 설정이 target에 자동으로 따라오면 새 환경을 오염시킬 수 있습니다. 그래서 런북은 운영 target에서 global state 자동 복원을 제한하고, 필요한 정책을 확인해서 명시적으로 적용합니다.

확인 대상	뜻
cluster health	cluster가 green인지, 문제 shard가 없는지
index count	문서 수가 source와 맞는지
mapping/settings	필드 구조와 index 옵션이 같은지
alias	애플리케이션이 보는 이름 연결이 같은지
template/pipeline	새 문서가 같은 방식으로 색인되는지
ILM/SLM	자동 삭제·rollover·snapshot 정책이 안전한지

11. ZooKeeper와 Solr: collection은 반드시 복원

Solr는 검색 서비스이고, ZooKeeper는 여러 Solr 노드가 서로 협력하도록 돕는 조정자입니다. Solr collection은 하나의 검색 공간이며, 내부에 shard, replica, leader, configset, alias가 함께 연결됩니다.

11.1 ZooKeeper를 새로 구성하는 이유

기존 ZooKeeper의 dataDir와 transaction log에는 예전 ensemble의 멤버십, epoch, ephemeral session 같은 동적인 정보가 들어 있습니다. 이 정보를 새 환경에 원시 복사하면 예전 클러스터와 새 클러스터가 섞일 수 있습니다. 그래서 런북은 clean ensemble을 만들고 필요한 설정만 다시 올립니다.

11.2 Solr collection 복원 흐름



12. MariaDB와 Scheduler: 가장 조심해야 하는 부분

MariaDB는 업무의 기준이 되는 데이터베이스입니다. Scheduler는 정해진 시간에 자동으로 DB, Solr, Redis, Engine 등을 바꿀 수 있습니다. 둘이 동시에 움직이면 source와 target의 데이터가 서로 달라집니다.

12.1 Scheduler를 먼저 멈추는 이유

- 새로운 통계·배포·학습 job이 계속 실행되면 백업 시점이 흔들린다.
- target에서 Scheduler를 켜면 source와 같은 job을 두 번 실행할 수 있다.
- 예약 배포 job은 DB뿐 아니라 Solr active/standby, Redis cache, Engine 알림까지 바꿀 수 있다.
- Quartz 설정에 중복 실행 방지 보장이 없으면 101, 102, 103에서 같은 job이 중복될 수 있다.

12.2 MariaDB 전체 복사와 선택 정합화

구분	처리 방향	이유
미러 DB	전체 database/schema/table 구조는 보존	VM 미러가 가진 실제 상태를 기준으로 사용
Scheduler 전용 통계·batch	source의 data-only INSERT SQL로 target 해당 데이터만 교체	Scheduler 영향 데이터의 시점을 맞춤
공용 업무 상태	영향받은 PK만 후보로 찾아 교정	CMS 시험 데이터까지 모두 지우면 안 됨
system DB, GRANT, DDL	자동으로 덮어쓰지 않음	버전·권한·환경 차이로 target을 망칠 수 있음

12.3 B0와 B1

이름	쉽게 말하면	목적
B0	첫 번째 연습용 백업	리허설과 복구 가능성 확인
B1	최종 전환 직전 백업	writer를 멈춘 뒤 최종 시점 동기화

핵심

MariaDB는 Redis처럼 '다 비우고 다시 만들면 된다'고 보면 안 된다. DB는 원본 업무 상태를 포함하므로, 어떤 table과 어떤 PK를 바꿀지 승인된 범위가 필요하다.

13. '복사'와 '복원'은 다르다

이번 이관을 이해하는 핵심입니다. VM을 복사한다고 해서 모든 서비스가 같은 방식으로 이관되는 것은 아닙니다.

대상	VM 복사로 따라오는 것	추가로 해야 하는 것
OS/프로그램	파일, 라이브러리, 설정	target OS/library 호환성과 기동 확인
CMS/Master 등 JAR	JAR와 설정 파일	target DB/Redis/Solr/ES 주소 확인
MariaDB	미러 data 디렉터리	무결성 확인과 선택 데이터 정합화
Redis	AOF/RDB/cluster state	재결합 또는 빈 cluster 재구성·cache 재생성
Elasticsearch	data 디렉터리 보관본	빈 cluster + Snapshot/Restore
Solr/ZK	data 디렉터리 보관본	clean ensemble + collection BACKUP/RESTORE
Qdrant	storage가 들어 있는 디스크	collection 목록과 snapshot/backup·대표 검색 확인
PostgreSQL	DB 파일 보관본	pg_dump/pg_restore 또는 승인된 복제와 권한 확인
Gitea	프로그램·저장소 파일·설정	DB, repositories, attachments, app.ini, 비밀번호를 일관되게 복원

따라서 이관 방식은 다음처럼 두 문장으로 말해야 정확합니다.

정확한 표현

VM 자체는 복사·이동 방식으로 target에 준비한다. 그 안의 서비스 데이터는 Redis, Elasticsearch, Solr, MariaDB의 성격에 맞는 별도 복원·정합화 절차를 적용한다.

'VM 복사 = 이관 완료'가 아닌 이유

- 복사본이 source와 같은 IP를 사용하면 두 서버가 동시에 보일 수 있다.
- 분산 서비스는 여러 노드의 시점과 cluster ID가 맞아야 한다.
- 애플리케이션 설정 안에 source IP나 운영 VIP가 남아 있을 수 있다.
- Scheduler가 target에서 자동 실행되면 업무 데이터가 변경될 수 있다.
- L4가 target으로 요청을 보내기 전까지는 실제 사용자가 target을 쓰지 않아야 한다.

13.1 목적지에 서비스가 이미 있으면 무엇이 달라지는가

기존 목적지 VM에 Qdrant, Elasticsearch, PostgreSQL, Gitea가 이미 있다면 두 가지 길이 생깁니다.

선택	무슨 뜻인가	위험과 필요한 확인
새 VM 복사	기존 목적지 VM은 그대로 두고 source VM을 새 이름·임시 IP로 복사	CPU·RAM·SR 여유, IP 충돌, 서비스가 source를 바라보는지 확인
서비스별 복원	기존 목적지 서비스는 유지하고 Qdrant·ES·DB·Gitea 데이터를 안전한 형식으로 넣음	기존 데이터 보존, 버전 호환, 백업·복원·다운타임 확인
기존 VM 교체	목적지 VM을 백업한 뒤 디스크나 VM을 교체	기존 서비스·데이터가 사라질 수 있어 별도 승인 필수

현재처럼 목적지에 이미 서비스가 있다는 정보가 있으면 ‘어떤 파일을 복사할까?’보다 먼저 ‘기존 목적지 서비스를 보존해야 하나?’를 결정해야 합니다. 이 결정 전에는 VM 디스크 덮어쓰기나 실행 중 데이터 폴더 복사를 하면 안 됩니다.

14. target 격리와 단계적 연결

target은 이관 후 새 환경을 뜻합니다. target을 처음부터 운영망에 연결하면 source와 target이 동시에 요청을 처리할 수 있어 사고가 납니다.

target VM 부팅
↓
source와 다른 임시 IP/hostname 또는 별도 포트그룹
↓
자동기동 차단과 source endpoint 접근 차단
↓
서비스별 복원과 내부 health 확인
↓
특정 테스트 IP 또는 격리된 L4 경로로 제한 요청
↓
오류율, 응답시간, 데이터 쓰기 관찰
↓
승인 후 target pool에 소량 또는 전체 전환

14.1 운영 트래픽을 마지막에 연결하는 이유

- 운영 사용자가 target의 미완성 데이터에 접속하는 것을 막는다.
- 문제가 나면 source를 유지한 채 요청을 source로 되돌릴 수 있다.
- 특정 IP만 연결하면 작은 범위에서 로그인, 검색, 배포를 시험할 수 있다.
- target이 source로 다시 쓰는 egress를 발견하기 쉽다.

14.2 target에서 특히 확인할 것

확인	왜 필요한가
DB/Redis/ZK/Solr/ES URL	target이 source를 보지 않게 하기 위해
hostname/IP/hosts	같은 이름의 서버가 충돌하지 않게 하기 위해
L4/VIP 미등록	운영 요청이 우연히 target으로 가지 않게 하기 위해
Scheduler OFF	중복 예약 작업을 막기 위해
외부 egress 차단	테스트 요청이 외부 시스템에 실제 작업을 만들지 않게 하기 위해

15. 개발을 공부하는 학생이 읽어야 하는 Linux 확인 명령어

아래 명령어는 대부분 조회용입니다. 명령어를 보면 '무엇을 확인하는지'를 먼저 이해해야 합니다.

명령어	뜻	이관에서 쓰는 이유
pwd	현재 위치 출력	영동한 디렉터리에서 작업하지 않는지 확인
ls -al	파일 목록과 권한 출력	JAR, conf, data 파일 존재 확인
find PATH -iname '*cms*'	파일 검색	CMS JAR와 설정 위치 찾기
grep -n '문자열' 파일	파일 안 문자열 검색	IP, port, URL, domain 찾기
ps auxf	실행 중인 프로세스 보기	CMS Java 프로세스가 있는지 확인
systemctl status NAME	systemd 서비스 상태 보기	자동기동 서비스 상태 확인
ss -ltnp	LISTEN 포트와 PID 보기	8280, 80, 9200 등이 열려 있는지 확인
ip addr	NIC와 IP 보기	VM 주소 확인
ip route	네트워크 경로 보기	gateway와 target route 확인
df -hT	파일시스템 용량 보기	target SR와 디스크 여유 확인
du -sh PATH	디렉터리 사용량 보기	data가 어디에 많이 있는지 확인
curl URL	HTTP 요청 보내기	서버 내부에서 API 응답 확인

15.1 명령어에서 자주 보는 기호

기호	뜻
sudo	관리자 권한으로 실행. 권한이 커지므로 목적을 확인
	앞 명령의 결과를 뒤 명령으로 넘김
>	결과를 파일에 새로 씀. 파일을 덮을 수 있어 주의
2>/dev/null	오류 메시지를 화면에서 숨김
*	여러 글자를 대신하는 와일드카드

16. '안 된다'를 상태별로 해석하는 법

관찰한 것	가능성이 높은 층	다음에 확인할 것
VM SSH 자체가 안 됨	VM/Network	IP, route, SSH daemon, firewall, source/target
ping은 되는데 TCP port 실패	Firewall/Port	해당 port LISTEN과 서버/중간 방화벽
서버 내부 curl은 401	Application/Auth	프로세스는 살아 있음. 로그인 경로와 계정 확인
IP 접속은 되는데 http://cms는 실패	hosts/HAProxy	hosts mapping과 Host ACL 확인
CMS 화면은 뜨는데 검색 실패	Solr/ES/Config	collection/index, URL, plugin, alias
로그인은 되는데 배포 실패	Master/Engine/DB/Redis	API 연결, DB 상태, cache와 job log
target 부팅 후 source 데이터가 바뀜	Isolation/Scheduler	writer, Scheduler, L4, egress 차단

이번 CMS 접속 문제에서의 실제 추론

Windows에서 192.168.1.102:8280은 TCP가 차단됐지만, 서버 내부 127.0.0.1:8280은 401을 반환했다. 이후 192.168.1.101:80의 HAProxy와 hosts의 cms 매핑을 찾아 http://cms 접속에 성공했다. 즉 'CMS가 죽었다'가 아니라 '직접 backend port가 아닌 기존 front door를 사용해야 했다'는 사례다.

17. 현재 환경에서 확인된 사실과 아직 모르는 것

이관 계획을 세울 때 확인된 사실과 추정 사항을 섞지 않아야 합니다. 아래 표는 이 문서의 기준입니다.

분류	내용
확인된 사실	192.168.1.101~103 VM에 SSH 접근 가능
확인된 사실	VM OS는 Rocky Linux 9.6, 각 VM은 약 5 vCPU/18 GiB/300 GiB 기준
확인된 사실	CMS JAR가 /application/cms/cms-0.0.1-SNAPSHOT.jar에 존재하고 한 노드에서 8280 LISTEN 확인
확인된 사실	192.168.1.101의 HAProxy가 0.0.0.0:80에서 대기하고 backend cms1/cms2/cms3:8280을 사용
확인된 사실	Host 이름 cms를 Windows hosts에서 192.168.1.101로 연결하면 http://cms 접속 가능
런북 기준	Elasticsearch는 Snapshot/Restore, Solr는 collection BACKUP/RESTORE 원칙
런북 기준	Redis는 기존 cluster 재결합 우선, 실패 시 빈 cluster와 cache 재생성
런북 기준	Scheduler는 마지막에 101 한 대만 제한 기동
확인된 자료	목적지 tc-xen01 아래에 xen01.01~03 VM과 192.168.1.81~83 주소가 기록됨
확인된 자료	목적지에는 Qdrant 01, Elasticsearch 9.1.1, PostgreSQL, Gitea가 기록됨
추가 확인 필요	xen01.01~03의 서비스가 현재 실제 사용 중인지와 담당자가 누구인지
추가 확인 필요	기존 목적지 VM을 보존할지, 새 VM을 만들지, 교체해도 되는지
추가 확인 필요	tc-xen01의 pool, SR, network, VM 생성·복사 권한
추가 확인 필요	Qdrant collection, Elasticsearch index, PostgreSQL DB, Gitea 저장소의 보존 범위
추가 확인 필요	meritz.easycms 등의 도메인이 브라우저용인지 내부 서비스용인지

18. 기초 용어 사전

용어	한 줄 설명
VM	물리 서버 안에서 실행되는 가상 컴퓨터
Host	VM을 실행하는 큰 컴퓨터
Pool	Xen 호스트들을 묶은 관리 그룹
SR	VM 디스크를 저장하는 창고
IP	서버의 네트워크 주소
Port	서버 안에서 프로그램을 구분하는 번호
Listen	프로그램이 해당 포트에서 연결을 기다리는 상태
Process	메모리에서 실행 중인 프로그램
systemd	Linux에서 서비스의 시작과 종료를 관리하는 기능
HAProxy	요청을 여러 backend에 나누어 보내는 프록시
Host header	HTTP 요청에 들어 있는 접속 이름
L4/VIP	여러 서버 중 어디로 보낼지 결정하는 네트워크 앞단
Cluster	여러 노드가 하나처럼 동작하는 묶음
Master/Replica	원본 역할과 복제본 역할
Shard	큰 검색 데이터를 나눈 조각
Collection	Solr에서 하나의 검색 공간
Index	Elasticsearch에서 검색할 문서 묶음
Snapshot/Restore	서비스가 이해하는 백업 형식으로 저장하고 복원
Cache	원본에서 다시 만들 수 있는 빠른 임시 데이터
Scheduler	예약 시간에 자동 업무를 실행하는 프로그램
Writer	DB나 검색 데이터에 실제로 쓰기를 하는 프로그램
Target	이관 후 새로 검증할 환경
Source	현재 운영 또는 원본 환경
Cut-over	최종적으로 요청을 새 환경으로 넘기는 전환
Rollback	문제가 생겼을 때 원본 환경으로 되돌리는 것

19. 이관이 성공했다는 뜻

'웹페이지가 떴다'만으로는 성공이 아닙니다. 아래 항목을 모두 확인해야 실제 이관 성공에 가까워집니다.

단계	성공 기준
VM/OS	target VM이 켜지고 CPU/RAM/디스크/OS가 기준과 일치
Network	target IP, gateway, DNS, route, hosts가 계획과 일치
Application	JAR checksum, 권한, profile, 로그 경로가 일치
DB	MariaDB와 PostgreSQL 무결성, 승인된 데이터 정합화, 비대상 table 불변
Redis	cluster state ok 또는 빈 cluster 재구성 후 cache 조회 정상
Solr/ZK	모든 collection, shard, replica, leader, alias, 한국어 검색 정상
ES/Kibana	snapshot restore 후 green, shard 0, index/mapping/alias/policy 정상
Qdrant/Gitea	collection 대표 검색, Gitea 로그인·저장소 clone/push-웹 화면 정상
Application chain	CMS -> Master -> Engine -> DB/Redis/Solr/ES 업무 흐름 정상
Scheduler	승인된 job만 101 한대에서 시험하고 중복/미종료 없음
Traffic	특정 테스트 경로 후 L4 전환, 오류와 source 접근 0 확인
Rollback	문제 발생 시 source를 보존하고 되돌릴 수 있음

쉬운 합격 문장

'프로그램 이름이 보인다'만으로는 부족하다. target이 실제 요청을 처리하고, 데이터가 맞고, 한 노드가 꺼져도 복구되며, 문제가 생기면 source로 돌아갈 수 있어야 이관 성공이다.

20. 지금 먼저 이해할 순서

모든 명령어를 한꺼번에 외울 필요는 없습니다. 아래 순서로 이해하면 됩니다.

1. tc-xen02는 원본 Host이고 101/102/103은 원본 VM이라는 것
↓
2. tc-xen01은 목적지 Host이고 xen01.01~03은 그 안의 기존 VM이라는 것
↓
3. CMS는 화면이고 Master/Engine/Gateway가 뒤에서 연결된다는 것
↓
4. http://cms는 Windows hosts와 HAProxy를 거쳐 CMS로 간다는 것
↓
5. 8280은 CMS backend port이고 80은 HAProxy front door라는 것
↓
6. Redis, ES, Solr, MariaDB, PostgreSQL, Qdrant가 서로 다른 데이터 성격을 가진다는 것
↓
7. 목적지에 기존 서비스가 있으면 덮어쓰지 않고 보존·새 VM·서비스 복원 중 하나를 결정한다는 것
↓
8. VM 복사 후 target을 격리해야 한다는 것
↓
9. 테스트 IP를 거쳐 마지막에 운영 L4로 전환한다는 것

팀장님께 설명할 때는 이렇게 말하면 됩니다.

한 문단 설명

이번 작업은 단순히 VM 디스크를 복사하고 IP만 바꾸는 작업이 아닙니다. VM 자체는 tc-xen01에 새 복사본으로 준비할 수 있지만, 목적지에 이미 있는 VM을 덮어써도 되는지 먼저 확인해야 합니다. 기존 서비스를 사용할 경우 Elasticsearch는 Snapshot/Restore, Solr는 collection BACKUP/RESTORE, Redis는 cluster 재결합 또는 재구성 후 cache 재생성, MariaDB와 PostgreSQL은 정합성을 지키는 백업·복원, Gitea는 DB·저장소·설정을 함께 복원합니다. target은 운영망과 분리해 검증하고 CMS와 시뮬레이터를 확인한 뒤 특정 IP 또는 L4를 통해 단계적으로 운영 전환합니다.

다음 문서와의 관계

문서	답을 내용
1번 이론	지금 문서. 구조와 원리를 이해
2번 계획	누가 언제 무엇을 어떤 승인으로 할지 결정
3번 실제 방법	명령어마다 목적, 예상 결과, 실패 시 다음 경로

끝.

21. 이 교재에서 다루는 실제 연습의 큰 그림

이제부터는 앞에서 배운 이론을 실제 연습 순서로 연결합니다. 이 단원은 '무엇을 복사하는가'와 '무엇을 새로 구성하는가'를 분리해서 생각하도록 돕습니다.

이번 연습을 한 문장으로

VM 2번의 서비스가 필요로 하는 라이브러리와 특정 디렉터리(logs, data 및 팀장님이 지정한 추가 디렉터리)를 VM 1번에 준비하고, 환경변수·설정·서비스 등록·클러스터 연결을 다시 맞춘 뒤 하나씩 실행하고 확인하는 연습입니다.

원본 VM 2번: 현재 서비스와 설정을 조사하고 필요한 것만 준비



공통 기반: Rocky Linux 9.6에서 필요한 라이브러리 설치



파일 이관: 지정된 디렉터리 압축 → 전송 → 목적지에서 해제



연결 구성: 환경변수·hosts·설정파일·권한·systemd 작성



데이터 구성: Redis/Solr-ZK/검색·DB의 역할에 맞게 복원 또는 재생성



목적지 VM 1번: 기존 서비스는 보존하되 Elasticsearch 겹침은 승인 후 정리



검증: 포트·로그·API·실제 시뮬레이터 기능을 순서대로 확인

이 흐름에서 '복사'라는 말은 파일을 옮기는 행동만 뜻하지 않습니다. 파일이 목적지에 있어도 프로그램이 그 파일을 어디서 읽을지 모르면 작동하지 않습니다. 그래서 이관은 '파일 준비 + 프로그램이 읽는 방법 설정 + 서로 연결 + 시험'의 네 가지가 함께 있어야 합니다.

22. VM 전체 복사와 서비스 재구성은 무엇이 다른가

방법	쉽게 말하면	이번 연습과의 관계
VM 전체 복사	컴퓨터 한 대를 통째로 복제한다. OS, 설치 프로그램, 사용자, 네트워크 설정, 디스크가 함께 따라온다.	이번 핵심 방식이 아님
서비스 재구성	빈 또는 기존 목적지 OS 위에 필요한 라이브러리·파일·설정·서비스를 다시 맞춘다.	이번 핵심 방식
데이터 복원	DB·검색엔진·벡터DB가 이해하는 백업 형식으로 데이터를 넣는다.	서비스별로 필요

예를 들어 'logs'를 복사하면 과거 실행 기록은 따라오지만, 로그 파일만으로 프로그램이 설치되거나 실행되지는 않습니다. 반대로 'data'를 복사하면 서비스가 저장한 상태가 따라올 수 있지만, 버전·권한·클러스터 정보가 맞지 않으면 데이터가 깨지거나 서비스가 시작되지 않을 수 있습니다. 따라서 '디렉터리만 압축하면 미러와 같다'는 말은 팀장님이 정한 전제, 즉 목적지에 필요한 라이브러리가 이미 설치되고 대상 디렉터리의 범위가 정확히 같다는 조건에서 이해해야 합니다.

중요한 안전 원칙

목적지 VM 1번에 이미 서비스가 있으므로 원본 파일을 무조건 덮어쓰지 않습니다. 먼저 백업·보존 범위·Elasticsearch 정리 범위를 확인하고, 같은 포트를 두 프로그램이 동시에 사용하지 않도록 합니다.

23. 팀장님이 준 dnf 명령어의 뜻

아래 명령은 Redis나 Elasticsearch를 설치하는 명령이 아닙니다. Java 프로그램과 압축·암호화·셸 도구가 실행될 수 있도록 Linux에 공통 부품을 설치하는 명령입니다.

```
sudo dnf install -y ₩
+ java-1.8.0-openjdk-headless ₩
+ glibc glibc-common glibc-langpack-en ₩
+ zlib libgcc libstdc++ libxcrypt-compat ₩
+ xz-libs systemd-libs libcap libgpg-error ₩
+ libzstd lz4-libs bash coreutils grep gawk sed ₩
+ procps-ng findutils ncurses-compat-libs
```

부분	뜻	왜 필요한가
sudo	관리자 권한으로 실행	일반 사용자가 시스템 패키지를 설치할 수 없기 때문
dnf	Rocky Linux의 패키지 관리자	인터넷 또는 사내 저장소에서 패키지를 찾아 설치
install	설치 작업	지정한 패키지를 OS에 추가
-y	질문에 자동으로 yes	중간 확인을 자동 승인하므로 운영 서버에서는 신중히 사용
₩	다음 줄도 같은 명령이라는 표시	줄을 나눠 읽기 쉽게 작성. 실제 입력 시 줄 끝에는 역슬래시 하나
java...	Java 8 실행 환경	Java 기반 CMS·Master·Engine·Solr·ZooKeeper 등 실행에 사용
glibc/libgcc/libstdc++	Linux 기본 실행 부품	프로그램이 OS 기능과 C/C++ 라이브러리를 사용하도록 지원
xz/zstd/lz4/zlib	압축 해제·압축 라이브러리	백업 파일, 로그, 데이터 압축 파일을 다룰 때 사용
bash/coreutils/grep/sed 등	조사·운영용 기본 명령 도구	프로세스, 설정, 로그, 파일을 확인하고 가공

기억할 문장

패키지는 레고 블록이고 서비스는 완성된 기계입니다. 레고 블록을 설치했다고 Redis·Solr·CMS가 자동으로 생기는 것은 아닙니다. 서비스 파일, 실행 파일, 설정, 데이터, systemd 등록을 별도로 준비해야 합니다.

24. 디렉터리·설정·환경변수는 각각 무슨 역할인가

종류	비유	확인해야 할 것
실행 파일/JAR	기계를 움직이는 본체	파일 위치, 버전, checksum, 실행 사용자
설정파일	기계의 사용 설명서	다른 서비스 주소·포트·경로·인증정보
logs	작업 일기장	과거 장애 분석용. 복사해도 과거 기록만 따라감
data	서비스가 보관한 실제 상태	DB·검색·캐시·벡터 데이터인지, 중지 후 복사해야 하는지
환경변수	프로그램에 건네는 메모	JAVA_HOME, DB 주소, Redis 주소, ES/Solr/ZK/Qdrant 주소
systemd unit	자동 출근 규칙	부팅 시 실행, 실행 사용자, 작업 경로, 재시작 정책
/etc/hosts	이름과 IP를 적은 전화번호부	cms, master, engine 같은 이름이 올바른 목적지로 가는지

환경변수 예시는 아래와 같습니다. 실제 값은 원본 VM을 조사해 얻어야 하며, 예시 값을 그대로 입력하면 안 됩니다.

```
JAVA_HOME=/usr/lib/jvm/jre-1.8.0-openjdkWn
DB_HOST=목적지-DB주소
DB_PORT=5432
REDIS_HOST=목적지-Redis주소
REDIS_PORT=7000
ELASTICSEARCH_URL=http://목적지-ES주소:9200
SOLR_URL=http://목적지-Solr주소:8983
ZK_HOST=목적지-ZK주소:2181
QDRANT_URL=http://목적지-Qdrant주소:6333
```

환경변수는 프로그램이 시작할 때 읽습니다. 따라서 값을 바꾼 뒤에는 이미 실행 중인 프로세스가 자동으로 새 값을 읽지 않습니다. 보통 설정 저장 → systemd daemon-reload(서비스 정의를 다시 읽기) → 해당 서비스 재시작 → 로그 확인 순서가 필요합니다. 비밀번호·토큰은 문서에 평문으로 남기지 말고 팀장님이 지정한 보안 보관 위치를 사용합니다.

25. 서비스 연결은 '주소를 서로 알려주는 일'이다

서비스는 혼자 일하는 프로그램이 아니라 서로 요청을 주고받는 프로그램입니다. CMS 화면에서 버튼을 누르면 CMS가 Master 또는 Gateway에 요청하고, 뒤에서는 DB·Redis·Solr·Elasticsearch·Qdrant가 데이터를 읽거나 씁니다.



왜 클러스터를 새로 잡는가

클러스터는 여러 프로그램이 '서로의 위치와 역할'을 기억하는 약속입니다. VM 주소가 바뀌면 예전 IP를 기억한 노드가 생길 수 있습니다. 그래서 파일을 옮긴 뒤에도 노드 목록, master/replica, shard/collection, leader, cluster name을 목적지 기준으로 다시 확인해야 합니다.

26. 이번 VM 2번 → VM 1번 연습 순서

아래는 실제 행동 문서로 이어질 수 있도록 만든 학습 순서입니다. 지금은 명령어를 외우기보다 각 단계의 질문을 이해하는 것이 먼저입니다.

순서	하는 일	이 단계에서 답해야 하는 질문
1	원본·목적지 식별	VM 2번의 주소는 무엇이고 VM 1번의 주소는 무엇인가?
2	원본 조사	어떤 서비스·포트·파일·라이브러리·데이터가 실제로 사용 중인가?
3	목적지 보호	xen01.01의 Qdrant 등 기존 서비스는 무엇이며 백업과 보존 방법은?
4	Elasticsearch 처리 결정	겉치레 ES의 프로그램·데이터·설정 중 어디까지 제거/보존하는가?
5	라이브러리 설치	dnf 명령으로 공통 실행 부품을 준비했는가?
6	디렉터리 전달	정확히 지정된 logs/data/추가 디렉터리만 옮겼는가?
7	권한·경로 정리	실행 사용자 admin이 파일을 읽고 쓸 수 있는가?
8	환경변수·설정 연결	목적지 주소가 들어갔고 원본 IP가 남아 있지 않은가?
9	기반 서비스 기동	DB·ZK·검색·Redis·Qdrant를 필요한 순서로 시작했는가?
10	애플리케이션 기동	Master → Engine/Gateway → CMS/Chat UI 순으로 확인했는가?
11	시뮬레이터 시험	학습 배포·봇 생성으로 Redis와 Solr collection 생성/사용을 확인했는가?
12	기능·로그·롤백 확인	실제 기능이 되고 오류가 없으며 되돌릴 수 있는가?

시뮬레이터에서 특히 기억할 점은 '모든 데이터를 미리 복사해야만 하는 것은 아니다'라는 것입니다. 팀장님 설명처럼 학습 배포 과정에서 Redis의 필요한 상태가 생성될 수 있고, 봇 생성 과정에서 Solr collection이나 관련 검색 구조가 만들어질 수 있습니다. 하지만 이것은 서비스가 정상 연결되어 있고 애플리케이션이 해당 생성 작업을 수행할 수 있을 때의 이야기입니다. 그러므로 '자동 생성된다'는 말은 백업이 필요 없다는 뜻이 아니라, 생성 가능한 데이터와 반드시 보존해야 하는 데이터를 구분하라는 뜻입니다.

27. 검증 명령어를 볼 때의 사고방식

2:3번 실습 문서에서는 명령어를 제시할 때 항상 네 가지를 함께 적습니다. 명령어만 복사하면 실수가 반복되므로, '무엇을 확인하는지'를 먼저 읽습니다.

항목	설명	예시 질문
명령어	Linux에 내리는 확인 또는 실행 지시	이 명령은 파일을 찾나, 포트를 보나?
기대값	정상일 때 화면에 나와야 하는 모습	LISTEN이 보여야 하나? active (running)이어야 하나?
해석	결과가 의미하는 상태	401이면 서버가 죽은 것이 아니라 인증이 필요한 것일 수 있나?
주의사항	실수하면 생길 수 있는 문제	rm·초기화·방화벽 변경인가? 승인 없이 해도 되나?
다음 행동	정상이 아니면 이어갈 확인	로그를 볼까, 설정 주소를 볼까, 팀장님께 물을까?

파일이 존재하는지 확인하는 예
ls -ld /application/cms

포트를 누가 사용 중인지 확인하는 예
sudo ss -lntp | grep ':8280'

서비스 상태를 읽기만 하는 예
systemctl status cms.service --no-pager

최근 로그를 읽는 예
journalctl -u cms.service -n 50 --no-pager

`ls -ld`는 파일/디렉터리의 존재와 권한을 봅니다. `ss -lntp`는 TCP 포트와 프로세스를 봅니다. `systemctl status`는 systemd 서비스가 실행 중인지 확인합니다. `journalctl`은 그 서비스가 왜 실패했는지 로그를 보여줍니다. 이 네 가지는 '파일이 있나 → 포트가 열렸나 → 서비스가 떴나 → 왜 실패했나'를 순서대로 확인하는 기본 도구입니다.

읽기 명령과 변경 명령을 구분하기

ls, ss, grep, systemctl status, journalctl은 대체로 확인용입니다. install, cp, tar 해제, systemctl enable/start, 방화벽 변경, 데이터 삭제·초기화는 상태를 바꿀 수 있으므로 계획과 승인 후 실행합니다.

28. 현재 목적지 환경에서 반드시 구분할 것

목적지 VM	기록된 기존 서비스	이번 연습에서의 원칙
xen01.01 / 192.168.1.81	Qdrant 01	기존 Qdrant를 함부로 삭제하지 않고 보존·백업·충돌 여부 확인
xen01.02 / 192.168.1.82	Elasticsearch 9.1.1 (9200/9300)	원본과 겹치므로 팀장님 승인 범위에 따라 제거 또는 재구성. 삭제 범위 확인 전 명령 금지
xen01.03 / 192.168.1.83	PostgreSQL 5432, Gitea 3000	DB·Gitea 데이터와 설정을 별도로 보존하고 원본 서비스와 연결 관계 확인

이 표에서 '기존 서비스가 있다'는 말은 '원본 서비스를 그 위에 무조건 덮어쓰다'는 뜻이 아닙니다. 먼저 목적지의 냉장고를 잠가 보관하듯 백업하고, 기존 서비스가 현재 사용 중인지 확인한 다음, 유지·데이터 교체·서비스 재설치 중 하나를 선택합니다.

다음 교재 파트에서는 각 선택에 따라 실제 명령을 작은 단계로 나누고, 각 단계에 정상 결과와 중단 기준을 붙입니다. 특히 Elasticsearch 삭제, DB 초기화, Redis cluster reset, Solr collection 삭제처럼 되돌리기 어려운 명령은 '실행 전 확인'으로 표시합니다.

29. 이 교재를 공부하는 방법

- 첫 번째 읽기: 명령어를 실행하지 말고 전체 흐름과 서비스 관계만 그림처럼 이해합니다.
- 두 번째 읽기: 원본 VM 2번과 목적지 VM 1번의 주소·서비스·포트를 표에 채웁니다.
- 세 번째 읽기: 각 명령어의 기대값을 먼저 예상한 뒤 실제 출력과 비교합니다.
- 네 번째 읽기: 실패하면 같은 명령을 반복하기보다 파일·권한·포트·주소·로그 중 어느 층에서 실패했는지 나눕니다.
- 마지막 읽기: 시뮬레이터에서 학습 배포·봇 생성·검색·저장까지 실제 업무 흐름을 확인합니다.

가장 중요한 결론

이번 연습의 목적은 한 번에 완벽한 운영 이관을 끝내는 것이 아니라, 원본을 조사하고 필요한 것만 목적지에 재구성하며, 서비스가 서로 연결되는 과정을 반복해서 이해하는 것입니다. 모르는 값을 추측해 실행하는 것보다 '이 값은 무엇이고 어디서 확인했는가'를 기록하는 습관이 더 중요합니다.

통합 교재의 나머지 실습 파트는 이 원칙에 맞춰 작성합니다. 원본 VM의 정확한 추가 디렉터리 이름, 라이브러리 전체 목록, 서비스별 설치 경로, Elasticsearch 제거 범위는 팀장님에게 받은 최종 자료를 기준으로 확정해야 합니다.

30. 현재 실행 방식: /application 스크립트와 MariaDB 수동 실행

팀장님이 알려준 현재 상태는 각 서비스의 시작·종료 스크립트가 이미 `/application` 아래에 배치되어 있다는 것입니다. 이번 연습에서는 서비스를 무조건 systemd로 새로 등록한다고 가정하지 않습니다. 먼저 기존 스크립트를 찾아서, 그 스크립트가 어떤 프로그램을 어떻게 실행하는지 읽고 사용합니다.

대상	현재 기준	실습에서 할 일
CMS·Master·Engine·Gateway·Chat UI 등	/application 아래 start/stop 스크립트 사용	스크립트 위치·실행권한·환경변수·로그경로·포트를 확인한 후 순서대로 실행
Redis·Solr·ZooKeeper·Elasticsearch·Qdrant	각 서비스별 기존 실행 방식 우선 확인	스크립트가 있으면 사용하고, 없으면 systemd/container 등 실제 관리방식을 조사
MariaDB	현재는 스크립트 없이 우선 수동 실행	데이터 디렉터리·소켓·포트·실행 사용자 확인 후 수동 기동

‘start 스크립트가 있다’는 것은 실행 버튼을 미리 만들어 두었다는 뜻입니다. 스크립트 안에는 Java 경로, JAR 위치, 설정파일 위치, 로그 위치, 메모리 옵션이 들어 있을 수 있습니다. 따라서 스크립트를 복사하는 것만으로 끝내지 말고 목적지 VM에서 그 안에 적힌 경로와 파일이 실제로 존재하는지 확인해야 합니다.

```
# /application 아래에서 시작·종료 스크립트 후보 찾기
find /application -maxdepth 4 -type f \
  \W( -iname '*start*' -o -iname '*stop*' -o -name '*.sh' ) \W \
  -print

# 실행 전 스크립트 내용 읽기
sed -n '1,220p' /application/서비스디렉터리/start.sh

# 실행권한 확인
test -x /application/서비스디렉터리/start.sh && echo '실행 가능' || echo '실행권한 없음'
```

확인 결과	뜻	다음 판단
start.sh와 stop.sh가 모두 있음	서비스를 켜고 끄는 절차가 준비됨	내용을 읽고 목적지 경로·주소를 확인한 뒤 사용
파일은 있지만 실행권한 없음	파일은 있으나 바로 실행할 수 없음	권한 변경 전 소유자·승인 확인
스크립트 안에 원본 IP가 있음	목적지가 원본으로 연결될 수 있음	목적지 주소로 바꿀지 설계 확인
실행 후 포트가 안 열림	실행 실패 또는 의존성 미충족 가능	로그·프로세스·포트·환경변수 순서로 조사

스크립트 실행 전 안전 규칙

start.sh를 읽기 전에 실행하지 않습니다. stop.sh는 현재 서비스를 끌 수 있고, 초기화·삭제 옵션이 들어 있을 수도 있습니다. 스크립트 안의 rm, kill, 데이터 경로, 포트, 원본 IP를 먼저 확인합니다.

MariaDB는 이번 기준에서 우선 수동 실행입니다. 이것은 중요하지 않다는 뜻이 아니라, 자동 시작 스크립트가 없는 상태에서 실행 과정과 정상 접속을 직접 확인한다는 뜻입니다. MariaDB의 데이터 디렉터리·소켓·포트·로그·실행 사용자를 먼저 확인한 뒤 다른 애플리케이션을 연결합니다.

1. /application에서 start/stop 스크립트 위치 찾기



2. 실행 전 스크립트 내용과 원본 IP·경로·포트 읽기

↓
3. 필요한 라이브러리와 지정 디렉터리 준비
↓
4. MariaDB는 승인된 방법으로 수동 기동·접속 확인
↓
5. ZooKeeper·검색·Redis·Qdrant 등 기반 서비스 확인
↓
6. Master·Engine·Gateway·CMS를 start 스크립트로 순차 기동
↓
7. 포트·로그·API·시뮬레이터 기능 검증

31. 실제 실습 명령어를 읽는 방법

이제부터 나오는 명령은 VM 2번을 '원본', VM 1번을 '목적지'라고 부릅니다. 아래 명령의 대괄호 부분은 실제 값을 넣어야 하는 자리입니다. 먼저 출력만 확인하는 명령부터 실행하고, 파일 이동·서비스 실행 명령은 팀장님이 정한 순서와 승인 후 실행합니다.

표시	뜻	예시
[VM2_IP]	원본 VM 2번의 실제 IP	192.168.x.x
[VM1_IP]	목적지 VM 1번의 실제 IP	192.168.1.81 등
[서비스명]	cms, master, redis 등 실제 이름	스크립트 폴더 이름과 일치시킴
[디렉터리]	팀장님이 지정한 실제 디렉터리	logs, data 및 추가 디렉터리

절대 그대로 복사하지 않기

`[VM2\_IP]`, `[서비스명]`, `[디렉터리]`는 설명용 표기입니다. 실제 값을 확인하지 않고 그대로 입력하면 명령이 실패하거나 엉뚱한 서버·폴더를 대상으로 할 수 있습니다.

32. 1단계 - VM 2번에서 원본 현황 조사

가장 먼저 하는 일은 '무엇을 옮길지 목록으로 만드는 것'입니다. 모르는 파일을 전부 압축하지 않고, 실행 중인 서비스가 실제로 어떤 파일을 사용하는지 확인합니다.

```
# 내가 접속한 VM의 정체 확인
```

```
hostname
```

```
hostname -I
```

```
cat /etc/os-release
```

```
# 디스크와 디렉터리 용량 확인
```

```
df -h
```

```
sudo du -sh /application/* 2>/dev/null
```

```
sudo du -sh /logs /data 2>/dev/null
```

```
# 실행 중인 프로세스와 열린 포트 확인
```

```
ps -ef --forest
```

```
sudo ss -lntp
```

```
# /application의 파일 목록 확인
```

```
sudo find /application -maxdepth 3 -type f -printf '%M %u %g %s %p\n' | sort
```

명령	무엇을 보는가	정상적으로 얻을 정보
hostname / hostname -I	서버 이름과 IP	VM 2번이 맞는지, 원본 주소가 무엇인지
cat /etc/os-release	Linux 배포판·버전	Rocky Linux 버전
df -h / du -sh	파일시스템·폴더 용량	압축할 공간과 전송할 데이터 크기
ps -ef	실행 중인 프로세스	Java-Redis-Solr 등 실제 실행 여부
ss -lntp	LISTEN 포트와 프로세스	서비스 포트와 충돌 여부
find /application	실행파일·스크립트 위치	start/stop, JAR, 설정파일 경로

기대값은 '명령이 성공했다'가 아닙니다. hostname이 VM 2번을 가리키고, `ss` 결과에 실제 서비스 포트가 보이며, `/application`에 start/stop 스크립트와 실행 파일이 보이는 것이 기대값입니다. 하나라도 다르면 압축하기 전에 조사 단계에서 멈춥니다.

33. 2단계 - VM 1번을 먼저 보호

목적지 VM 1번에는 Qdrant-Elasticsearch-PostgreSQL-Gitea처럼 이미 사용 중인 서비스가 있습니다. 따라서 원본을 옮기 전에 목적지의 상태를 기록하고 백업 여부를 확인합니다.

```
# 반드시 VM 1번에 접속한 뒤 실행
hostname
hostname -l
df -h
sudo ss -lntp
ps -ef --forest

# 기존 /application과 지정 데이터의 목록만 확인
sudo du -sh /application/* 2>/dev/null
sudo du -sh /logs /data 2>/dev/null

# 서비스 관리자 등록 여부 확인
systemctl list-unit-files --type=service | grep -Ei 'redis|elastic|solr|zookeeper|qdrant|postgres|gitea|mariadb'
```

Elasticsearch 주의

팀장님 기준으로 VM 1번의 Elasticsearch는 원본과 겹치므로 정리 대상이 될 수 있습니다. 그러나 `systemctl disable`, `dnf remove`, 데이터 디렉터리 삭제, cluster 초기화는 서로 다른 작업입니다. 무엇을 보존하고 어디까지 지울지는 승인받기 전까지 실행하지 않습니다.

목적지 조사의 기대값은 '기존 냉장고의 내용물을 확인했다'는 것입니다. 기존 서비스의 포트·데이터 경로·실행 사용자를 기록해야 이후 원본 파일을 어디에 넣을지 결정할 수 있습니다.

34. 3단계 - 공통 라이브러리 준비

원본에서 사용하는 Java와 Linux 공통 라이브러리를 목적지에 설치합니다. 이 명령은 애플리케이션 서비스를 설치하는 명령이 아니라 실행에 필요한 기반 부품을 설치하는 명령입니다.

```
sudo dnf install -y \
+ java-1.8.0-openjdk-headless \
+ glibc glibc-common glibc-langpack-en \
+ zlib libgcc libstdc++ libxcrypt-compat \
+ xz-libs systemd-libs libcap libgcrypt libgpg-error \
+ libzstd lz4-libs bash coreutils grep gawk sed \
+ procps-ng findutils ncurses-compat-libs
```

```
java -version
rpm -qa | grep -E 'java-1.8|glibc|zlib|libstdc++|libgcc'
```

기대 결과	해석
dnf가 완료되고 오류 없음	패키지 설치가 끝남. 그래도 서비스 설치 완료는 아님
java -version이 1.8 계열	Java 기반 프로그램을 실행할 기반이 준비됨
No match for argument	저장소·패키지명 문제 가능. 임의로 다른 버전 설치하지 말고 확인
패키지 충돌 또는 의존성 오류	OS 상태를 바꾸기 전에 출력 내용을 저장하고 중단

`sudo`는 관리자 권한, `dnf`는 Rocky Linux 패키지 도구, `-y`는 설치 확인 질문에 자동으로 동의한다는 뜻입니다. 운영 서버에서는 `-y`를 사용하기 전에 설치 목록과 대상 VM을 다시 확인합니다.

35. 4단계 - /application 스크립트 확인

팀장님이 알려준 현재 구조에서는 서비스 시작·종료 스크립트가 `/application` 아래에 있습니다. 따라서 먼저 스크립트를 찾고 읽은 다음, 필요한 디렉터리와 설정을 준비합니다.

```
sudo find /application -maxdepth 4 -type f \\  
+ \$( -iname '*start*' -o -iname '*stop*' -o -name '*.sh' ) -print  
  
# 실제 찾은 파일로 바꿔서 읽기  
sed -n '1,220p' /application/[서비스]/[start스크립트].sh  
sed -n '1,220p' /application/[서비스]/[stop스크립트].sh  
  
# 스크립트가 참조하는 경로·주소·포트 찾기  
grep -nEi 'java|jar|config|log|data|host|port|redis|solr|elastic|zookeeper|maria|postgres|qdrant' \\  
/application/[서비스]/[start스크립트].sh
```

확인 항목	왜 필요한가
JAVA_HOME:java 경로	목적지의 Java 위치가 원본과 다르면 실행 실패
JAR·설정파일 경로	파일을 정확한 위치에 배치해야 함
로그 경로	실행 후 오류를 확인할 위치
원본 IP·서비스 이름	목적지 서비스 주소로 바꿔야 하는 값
메모리 옵션	VM 1번의 RAM에 맞는지 확인
실행 사용자	파일을 읽고 로그를 쓸 권한이 있는지 확인

실행 전 중단 기준

스크립트 안에 데이터 삭제, 강제 종료, 원본 IP, 비밀번호, 다른 VM 주소가 보이는데 의미를 모르면 실행하지 않습니다. 내용을 팀장님과 확인한 뒤 목적지용 값으로 정합니다.

36. 5단계 - 지정 디렉터리 압축과 전송

이번 작업은 팀장님이 지정한 디렉터리만 옮깁니다. 아래 예시는 `logs`, `data`, 그리고 확인된 추가 디렉터리를 `[추가디렉터리]` 자리에 넣는 방식입니다. 실제 이름을 확정하기 전에는 압축하지 않습니다.

```
# VM 2번에서: 서비스 중지 여부를 먼저 확인한 뒤 압축
sudo tar -czpf /tmp/vm2_service_20260904.tar.gz \w
/logs /data /[추가디렉터리]

# 압축 파일 내용 확인 (해제하지 않고 목록만 확인)
tar -tzf /tmp/vm2_service_20260904.tar.gz | head -50

# 파일 크기와 checksum 기록
ls -lh /tmp/vm2_service_20260904.tar.gz
sha256sum /tmp/vm2_service_20260904.tar.gz

# VM 2번에서 VM 1번으로 전송 (주소와 계정은 실제 값으로 교체)
scp /tmp/vm2_service_20260904.tar.gz admin@[VM1_IP]:/tmp/
```

명령	뜻	주의
tar -czpf	여러 파일을 하나의 gzip 압축파일로 묶음	경로를 잘못 쓰면 다른 데이터가 포함될 수 있음
tar -tzf	압축을 풀지 않고 내부 목록 확인	전송 전 대상 목록 검증
sha256sum	파일 지문 계산	원본과 목적지 지문이 같아야 전송 중 변조·손상 없음
scp	SSH를 통해 파일 복사	목적지 IP·계정·저장 경로를 확인

실행 중인 DB나 검색엔진의 데이터 디렉터리를 무조건 tar로 복사하면 일관성이 깨질 수 있습니다.

MariaDB·Redis·Solr·Elasticsearch·Qdrant 데이터는 서비스별 백업·중지 조건을 먼저 확인합니다. `logs`는 기록이고 `data`는 실제 상태이므로 같은 방식으로 취급하지 않습니다.

37. 6단계 - 목적지에서 해제·권한·환경변수 준비

```
# VM 1번에서: 전송 파일의 checksum 확인
sha256sum /tmp/vm2_service_20260904.tar.gz
```

```
# 원본과 같은지 비교한 뒤, 내부 목록 재확인
tar -tzf /tmp/vm2_service_20260904.tar.gz | head -50
```

```
# 목적지의 작업 공간에 해제 (경로는 승인된 위치로 변경)
sudo tar -xzf /tmp/vm2_service_20260904.tar.gz -C /
```

```
# 소유자와 권한 확인
sudo find /application /logs /data -maxdepth 2 -type d -ls 2>/dev/null
```

```
# 환경변수 파일 위치 후보 확인
find /application -maxdepth 4 -type f \
  \W( -name '*.env' -o -name '*.properties' -o -name '*.yml' -o -name '*.yaml' \W) -print
```

`tar -xzf -C /`는 압축파일에 기록된 절대경로를 루트(`/) 아래에 복원하는 예입니다. 경로가 틀리면 파일을 덮어쓸 수 있으므로 목록과 대상 경로를 확인한 뒤 사용합니다. 목적지에 이미 같은 폴더가 있으면 먼저 백업하고, 단순 해제가 기존 파일을 덮어쓰는지 확인해야 합니다.

확인	기대값
checksum	VM 2번과 VM 1번 출력이 동일
소유자·권한	실제 실행 사용자(admin 등)가 읽기·쓰기 가능
경로	start 스크립트가 기대하는 위치에 파일 존재
환경변수	원본 주소가 아니라 목적지 서비스 주소를 가리킴

38. 7단계 - 서비스 연결과 순차 기동

서비스를 한꺼번에 켜지 않고 아래 순서로 한 개씩 실행합니다. 앞 단계가 정상이어야 뒤 단계가 의미 있는 오류를 낼 수 있습니다.

MariaDB: 수동 기동 → 로컬 접속·포트·로그 확인
↓
ZooKeeper: ensemble-2181 확인
↓
Elasticsearch/Solr/Redis/Qdrant: 상태·포트·데이터 확인
↓
Master: DB·Redis·검색 주소 확인
↓
Engine/Gateway: Master와 외부 연결 확인
↓
CMS/Chat UI: start 스크립트로 기동 → 8280/8480 확인
↓
Scheduler: 마지막에 승인된 한 대만 제한 기동
↓
Simulator: 학습 배포·봇 생성·검색·저장 테스트
서비스별 start 스크립트 실행 예 cd /application/[서비스디렉터리] ./start.sh # 실행 결과 확인 ps -ef grep -v grep grep -i '[서비스명]' sudo ss -lntp grep -E ':(포트번호)\\b' tail -n 100 /logs/[서비스로그].log # 종료가 필요한 경우에만 승인 후 실행 ./stop.sh
MariaDB는 수동 실행
MariaDB는 현재 start/stop 스크립트가 없으므로 우선 담당자가 정한 수동 실행 명령과 절차를 사용합니다. 실행 방법을 추측하지 말고 데이터 디렉터리, socket, 포트, 실행 사용자, 로그 위치를 확인한 뒤 진행합니다.

39. 8단계 - 정상인지 확인하는 네 겹의 검증

검증 층	질문	예시
프로세스	프로그램이 메모리에서 실행 중인가?	ps -ef, systemctl status
포트	연결을 기다리고 있는가?	ss -lntp에서 LISTEN
응답	요청을 받아 정상 응답하는가?	curl, 서비스 상태 API
기능	실제 업무가 끝까지 되는가?	CMS 로그인·학습 배포·봇 생성·검색

CMS backend가 목적지 내부에서 응답하는지 확인하는 예
curl -i --max-time 5 http://127.0.0.1:8280/

결과 해석 예

200: 인증 없이 정상 응답할 수 있음

401: 서버는 살아 있고 인증이 필요할 수 있음

connection refused: 프로세스 또는 포트 문제

timeout: 네트워크·방화벽 주소 문제 가능

HTTP 401은 '서비스가 죽었다'는 뜻이 아닙니다. 서버가 요청을 받았지만 로그인이나 인증이 필요하다는 뜻일 수 있습니다. 반대로 포트가 LISTEN이어도 내부 DB 연결이 끊겼다면 실제 기능은 실패할 수 있으므로 반드시 시뮬레이터까지 확인합니다.

최종 합격 기준

VM 1번에서 서비스 프로세스가 실행되고, 포트가 열리고, API가 응답하고, CMS에서 실제 업무 흐름이 성공하며, Redis·검색·DB 데이터가 의도대로 연결되고, 문제가 생기면 VM 2번 원본으로 돌아갈 수 있어야 합니다.

40. 실패했을 때 조사 순서

1. 파일이 있는가? → <code>ls/find/du</code>
↓
2. 권한이 맞는가? → <code>ls -l/namei</code>
↓
3. 환경변수·주소가 맞는가? → <code>grep/sed</code> 로 설정 읽기
↓
4. 프로세스가 켜있는가? → <code>ps/systemctl</code>
↓
5. 포트가 열렸는가? → <code>ss</code>
↓
6. 로그에 첫 번째 ERROR는 무엇인가? → <code>tail/journalctl</code>
↓
7. 의존 서비스가 먼저 정상인가? → <code>DB/Redis/Solr/ZK/ES/Qdrant</code>
↓
8. 데이터 구조가 맞는가? → <code>index/collection/schema/cluster</code> 확인

실패한 명령을 계속 반복하는 것보다, 실패가 파일·권한·설정·프로세스·포트·데이터 중 어느 층인지 좁히는 것이 빠릅니다. 로그의 마지막 줄만 보지 말고 최초 오류를 찾습니다. 같은 문제가 세 번 반복되면 명령을 멈추고 출력·시각·대상 VM을 기록해 팀장님께 공유합니다.

상황	우선 확인할 것	하지 말아야 할 것
포트가 안 열림	start 스크립트 출력·로그·ps	방화벽부터 무작정 변경
검색 결과 없음	Solr collection/ES index/alias 주소	검색 데이터를 바로 삭제하고 재생성
Redis 오류	cluster node/slot-host/port	cluster reset을 승인 없이 실행
DB 연결 실패	DB 프로세스·계정·socket·포트	DB 디렉터리 덮어쓰기

41. 명령어 하나씩 해부하기

이 단원은 앞에서 나온 명령어를 '통째로 외우지 않도록' 쪼개서 설명합니다. Linux 명령은 보통 [명령어] [옵션] [대상] 구조입니다. 앞에 `sudo`가 붙으면 관리자 권한으로 실행한다는 뜻이고, `|`는 앞 명령의 결과를 뒤 명령에 넘긴다는 뜻입니다.

명령어	부분별 뜻	전체 의미
hostname	hostname = 서버 이름을 보여주는 명령	내가 어느 VM에 접속했는지 확인
hostname -I	-I = IP 주소만 표시	현재 VM의 네트워크 주소 확인
cat /etc/os-release	cat = 내용 출력, /etc/os-release = OS 정보 파일	Rocky Linux 버전 확인
df -h	df = 디스크 사용량, -h = 사람이 읽기 쉬운 단위	파일시스템 전체 용량 확인
du -sh /logs	du = 폴더 사용량, -s = 합계만, -h = GB/MB 단위, /logs = 대상	/logs가 차지하는 전체 용량 확인
sudo du -sh /data	sudo = 관리자 권한, du -sh = 폴더 합계 용량, /data = 대상	권한이 필요한 data 용량 확인
ps -ef	ps = 프로세스, -e = 모든 프로세스, -f = 자세한 형식	실행 중인 프로그램과 실행 사용자 확인
sudo ss -lntp	ss = 네트워크 소켓, -l = LISTEN만, -n = 숫자로, -t = TCP, -p = 프로세스 표시	포트와 그 포트를 쓰는 프로세스 확인
grep -Ei 'redis solr'	grep = 문장 검색, -E = 정규식, -i = 대소문자 무시	출력 중 Redis 또는 Solr 관련 줄만 선택

예를 들어 `sudo du -sh /logs`를 읽을 때는 'sudo로 관리자 권한을 얻고, du로 디스크 사용량을 계산하며, -s로 하위 파일을 하나씩 보여주지 않고 합계만 보고, -h로 사람이 읽기 쉬운 단위로 표시하고, /logs 폴더를 대상으로 한다'고 해석합니다.

42. 파일 찾기·읽기 명령어 해부

명령어	부분별 뜻	기대 결과
find /application -maxdepth 4 -type f -print	find = 찾기, /application = 시작 위치, -maxdepth 4 = 최대 4단계, -type f = 일반 파일, -print = 경로 출력	start/stop-JAR-설정파일 목록
sed -n '1,220p' file	sed = 텍스트 처리, -n = 지정 줄만 출력, '1,220p' = 1~220줄, file = 파일	스크립트 앞부분을 변경 없이 읽기
ls -l	ls = 목록, -l = 권한·소유자·크기·시간을 자세히	파일 권한과 소유자 확인
ls -ld /application	-d = 폴더 안이 아니라 폴더 자체를 표시	디렉터리 자체의 권한 확인
test -x file && echo ... echo ...	test = 조건 검사, -x = 실행 가능 여부, && = 성공 시 다음, = 실패 시 다음	스크립트 실행권한 확인
grep -nEi 'java port' file	-n = 줄 번호, -E = 여러 검색어, -i = 대소문자 무시, = OR	설정·스크립트에서 관련 줄 찾기

```
find /application -maxdepth 4 -type f ₩
+ ₩(-iname '*start*' -o -iname '*stop*' -o -name '*.sh' ₩) -print
```

위 명령에서 ``-o``는 OR, ``-iname``은 대소문자를 무시한 이름 검색, ``*start*``는 이름 중간에 start가 들어간 파일을 뜻합니다. ``₩``(와 ``₩``)는 여러 조건을 하나의 묶음으로 묶는 표시입니다. 즉 `/application` 아래 4단계 안에서 이름에 start 또는 stop이 들어가거나 확장자가 .sh인 일반 파일을 찾아 출력하는 명령입니다.

43. 압축·전송 명령어 해부

명령어	부분별 뜻	주의
<code>tar -czpf archive.tar.gz /logs /data</code>	tar = 묶기, -c = 생성, -z = gzip, -p = 권한 보존, -f = 다음 이름을 파일명으로 사용	대상 경로를 잘못 쓰지 않기
<code>tar -tzf archive.tar.gz</code>	-t = 목록, -z = gzip 해제 방식, -f = 파일 지정	압축을 풀지 않고 내부 목록만 확인
<code>tar -xzpf archive.tar.gz -C /</code>	-x = 풀기, -z = gzip, -p = 권한 보존, -f = 파일 지정, -C / = 루트에서 해제	기존 파일을 덮어쓸 수 있으므로 승인 필요
<code>sha256sum archive.tar.gz</code>	sha256sum = 파일 지문 계산	원본·목적지 값이 같아야 함
<code>scp file admin@[VM1_IP]:/tmp/</code>	scp = SSH 파일 복사, file = 원본, admin = 목적지 사용자, :/tmp = 목적지 경로	IP·계정·경로 확인

`tar -czpf`에서 c는 create(생성), z는 gzip 압축, p는 permission(권한) 보존, f는 file(파일명 지정)입니다. 반대로 `tar -xzpf`에서 x는 extract(압축 해제)입니다. `scp`는 파일을 복사하지만 서비스 연결·환경변수·클러스터를 자동으로 맞춰주지는 않습니다.

44. 서비스·로그·HTTP 명령어 해부

명령어	부분별 뜻	무엇을 판단하는가
systemctl status service --no-pager	systemctl = 서비스 관리자, status = 상태 확인, --no-pager = 화면 넘김 없이 끝까지	서비스가 active인지·실패 이유가 있는지
journalctl -u service -n 50 --no-pager	journalctl = systemd 로그, -u = unit 지정, -n 50 = 최근 50줄	해당 서비스의 최근 오류
tail -n 100 logfile	tail = 파일 끝부분, -n 100 = 마지막 100줄	최근 애플리케이션 로그
curl -i --max-time 5 http://127.0.0.1:8280/	curl = HTTP 요청, -i = 응답 헤더 포함, --max-time 5 = 5초 제한	CMS가 HTTP 요청에 응답하는지
cd /application/service	cd = 작업 디렉터리 이동	이후 상대경로 명령의 기준 위치 변경
./start.sh	./ = 현재 폴더, start.sh = 스크립트 실행	현재 서비스 시작

`systemctl status`가 active라고 해도 실제 기능까지 보장하지는 않습니다. 포트, 로그, API, 시뮬레이터를 차례로 확인해야 합니다. `curl`의 401은 서버가 요청을 받았지만 인증이 필요하다는 뜻일 수 있고, connection refused는 보통 해당 포트에 듣고 있는 프로세스가 없다는 뜻입니다.

45. dnf와 패키지 확인 명령어 해부

부분	뜻	이번 작업에서의 역할
sudo	관리자 권한으로 실행	시스템 패키지 설치에 필요
dnf	Rocky Linux 패키지 관리자	저장소에서 패키지를 찾고 설치
install	설치 동작	지정 라이브러리 추가
-y	확인 질문에 자동 yes	자동 실행 옵션이므로 대상 확인 필요
₩	명령이 다음 줄에도 계속됨	실제 줄 끝에는 역슬래시 하나
rpm -qa	설치된 모든 RPM 패키지 조회	현재 설치 상태 확인
rpm -qa grep 'java'	rpm 결과를 grep으로 필터링	Java 패키지 설치 여부 확인

'sudo dnf install -y java-1.8.0-openjdk-headless'는 '관리자 권한으로 dnf를 사용해 install 작업을 하고, -y로 확인 질문에 자동 동의하며, Java 8의 화면 없는(headless) 실행 환경을 설치하라'는 뜻입니다. 이것은 CMS-Redis-Solr 같은 서비스 자체를 설치하는 명령이 아닙니다.

명령어 학습 규칙

앞으로의 모든 실습 명령은 명령어, 토큰별 뜻, 기대 결과, 결과 해석, 주의사항, 다음 행동의 순서로 읽습니다. 이해되지 않는 토큰이 하나라도 있으면 실행하지 않고 먼저 뜻을 확인합니다.

46. 초보자용 실습 방식 - 디렉터리를 직접 찾아가기

실제 작업에서는 어려운 명령을 한 줄로 외우지 않습니다. 먼저 내가 어디 있는지 확인하고, 눈으로 폴더를 보고, 한 단계씩 들어갑니다. 아래 명령은 VM 2번과 VM 1번에서 같은 방식으로 반복합니다.

1. 서버에 접속한다
↓
2. 내가 어느 서버·어느 폴더에 있는지 확인한다
↓
3. /application 폴더로 이동한다
↓
4. 서비스 폴더 이름을 눈으로 확인한다
↓
5. 서비스 폴더 안으로 들어간다
↓
6. start/stop, JAR, 설정파일 을 하나씩 확인한다
↓
7. 필요한 파일과 디렉터리만 기록한다

첫 단계는 아무것도 바꾸지 않는 확인입니다.

```
pwd
hostname
hostname -I
ls
```

명령	아주 쉽게 말하면	기대 결과
pwd	현재 내가 들어와 있는 폴더 주소를 보여줘	예: /home/admin
hostname	지금 접속한 서버 이름을 보여줘	VM 2번인지 VM 1번인지 확인
hostname -I	이 서버의 IP를 보여줘	원본·목적지 IP 기록
ls	현재 폴더 안에 무엇이 있는지 보여줘	폴더와 파일 이름 목록

'pwd'는 '내가 지금 어느 방에 있는가'를 확인하는 명령입니다. 'hostname'과 'hostname -I'는 '어느 컴퓨터에 있는가'를 확인합니다. 이 세 가지를 먼저 입력하면 다른 VM에서 실수로 작업하는 것을 줄일 수 있습니다.

47. /application 안을 직접 확인하기

이제 서비스가 모여 있는 `/application`으로 이동합니다. `/`는 Linux 파일시스템의 가장 위이고, `/application`은 그 아래에 있는 폴더입니다.

```
cd /application
pwd
ls
ls -l
```

`cd /application`은 `/application` 폴더로 이동하라는 뜻입니다. `cd`는 change directory, 즉 디렉터리 이동입니다. 이동한 뒤 `pwd`를 다시 입력해 정말 `/application`에 왔는지 확인합니다. `ls -l`은 파일 이름뿐 아니라 권한·소유자·크기까지 보여줍니다.

목록에 서비스 이름처럼 보이는 폴더가 있으면 한 번에 모두 처리하지 말고 하나씩 들어갑니다.

```
cd /application/[서비스폴더명]
pwd
ls
ls -l
```

보이는 것	의미	다음 행동
start.sh	서비스 시작용 스크립트	내용을 읽고 실행 방법 확인
stop.sh	서비스 종료용 스크립트	삭제·kill 내용이 있는지 먼저 읽기
*.jar	Java 프로그램 본체	파일명·버전·크기 기록
conf/config/application.yml	서비스 설정파일 후보	주소·포트·데이터 경로 확인
logs 또는 log	서비스 로그 폴더 후보	실행 후 오류 확인 위치 기록

폴더 이름은 예시와 다를 수 있음

`[서비스폴더명]`을 그대로 입력하지 않습니다. `ls` 화면에서 실제 이름을 보고 그 이름으로 다시 입력합니다. Linux는 대문자와 소문자를 다른 이름으로 취급합니다.

48. 파일 하나씩 읽어보기

파일을 수정하기 전에 먼저 읽습니다. `cat`은 짧은 파일을 한 번에 보여주고, `less`는 긴 파일을 위아래로 천천히 볼 때 사용합니다.

```
cat start.sh
less start.sh

# less 화면에서 나가기
q
```

명령	뜻	언제 사용하나
cat start.sh	start.sh의 전체 내용을 화면에 출력	짧은 파일
less start.sh	파일을 한 화면씩 읽기	긴 스크립트·로그
q	less 화면에서 나가기	읽기를 끝낼 때
grep 'java' start.sh	start.sh 안에서 java가 있는 줄만 찾기	필요할 때만 사용

스크립트에서 다음 내용을 눈으로 찾습니다: 실행할 JAR 파일, Java 경로, 설정파일, 로그 경로, 메모리 옵션, 다른 서비스의 주소와 포트입니다. 예를 들어 `java -jar`가 보이면 Java로 JAR 프로그램을 실행한다는 뜻이고, `--spring.config.location`이 보이면 별도 설정파일을 읽는다는 뜻일 수 있습니다.

수정은 읽기가 끝난 뒤

처음에는 `vi`, `sed -i`, `cp` 같은 변경 명령을 사용하지 않습니다. 원본과 목적지의 내용을 비교하고, 바꿀 값과 백업 위치를 정한 다음에만 수정합니다.

49. 하나의 서비스만 시험하는 실제 순서

서비스 하나를 예로 들면 다음처럼 진행합니다. `[서비스폴더명]`과 `[start파일명]`은 `ls`로 확인한 실제 이름으로 바꿉니다.

```
cd /application
ls
cd /application/[서비스폴더명]
pwd
ls -l
cat [start파일명]

# 실행 전 현재 프로세스·포트 확인
ps -ef
sudo ss -lntp

# 승인 후에만 시작
./[start파일명]

# 시작 후 확인
ps -ef
sudo ss -lntp
ls
less /logs/[로그파일명]
```

순서	왜 하는가	정상이면
cd:pwd	올바른 서비스 폴더인지 확인	목적지 경로가 맞음
ls -l	스크립트와 파일 존재·권한 확인	파일이 있고 실행권한이 있음
cat start파일	실행 내용을 미리 읽음	주소·경로·포트가 이해됨
ps:ss	시작 전 상태 기록	기존 충돌 포트 없음
./start파일	서비스 시작	스크립트가 오류 없이 끝남
ps:ss:로그	실제로 댔는지 확인	프로세스·LISTEN·정상 로그

이 방식의 핵심은 ‘start 명령을 입력했으니 성공’이라고 생각하지 않는 것입니다. 시작 전 상태와 시작 후 상태를 비교해야 합니다. 포트가 이미 다른 프로그램에 사용 중이면 시작하지 말고 먼저 충돌을 해결합니다.

50. 파일 수정도 한 줄씩 천천히 하기

목적지 VM 1번에서는 원본 VM 2번의 주소를 그대로 쓰면 안 될 수 있습니다. 하지만 어떤 주소를 바꿀지는 설계가 확정된 뒤 결정합니다. 수정 전에는 원본을 백업합니다.

```
# 수정 전 파일을 다른 이름으로 보관하는 예
cp application.properties application.properties.before-change
```

```
# 파일 내용 확인
cat application.properties
```

```
# 특정 단어가 있는 줄 찾기
grep 'redis' application.properties
grep 'solr' application.properties
grep 'elasticsearch' application.properties
```

명령	부분별 뜻	주의
cp source backup	cp = copy, source = 원본, backup = 백업 이름	백업 파일이 실제로 생겼는지 ls로 확인
grep 'redis' file	file 안에서 redis라는 글자가 있는 줄 검색	비밀번호가 화면에 보일 수 있으므로 출력 공유 주의
cat file	파일 전체 출력	비밀번호·토큰이 있으면 화면 캡처 금지

설정 파일을 수정할 때는 '원본 값 → 목적지 값 → 변경 이유'를 메모합니다. 예를 들어 Redis 주소를 바꾸는 것은 Redis 프로그램을 설치하는 일이 아니라, 이 서비스가 어느 Redis에 연결할지 알려주는 일입니다. 바꾼 뒤에는 해당 서비스만 재시작하고 로그로 새 주소를 읽었는지 확인합니다.

51. 이 교재의 새 실행 규칙

- 긴 명령어를 바로 실행하지 않고 먼저 현재 위치와 서버 이름을 확인합니다.
- `cd`로 폴더에 들어간 뒤 `pwd`와 `ls`로 실제 경로와 파일을 확인합니다.
- 스크립트는 실행 전에 `cat` 또는 `less`로 내용을 읽습니다.
- 파일을 수정하기 전에는 `cp`로 백업하고, 바뀐 값과 이유를 기록합니다.
- 서비스는 하나씩 시작하고, 프로세스·포트·로그를 각각 확인합니다.
- MariaDB는 현재 기준으로 자동 스크립트가 아닌 수동 실행 절차를 따릅니다.
- 삭제·초기화·클러스터 리셋은 설명서에 표시된 '승인 필요' 단계로 남깁니다.

지금 실제로 해야 할 첫 연습

VM 2번에 접속해 `hostname`, `hostname -I`, `pwd`, `ls`를 입력하고, `/application`으로 이동해 `ls -l`을 확인합니다. 그 결과를 기록한 뒤 서비스 폴더 하나에 들어가 start/stop 파일과 JAR·설정파일 이름을 확인하는 것부터 시작합니다.